

The Butterfly Effect of SGD Noise: Why Your Reward Swings While Your Loss Stays Flat

ICLR 2024 · MIT and Microsoft Research trace long-horizon instability in behavior cloning and language models to optimizer noise — and show that averaging your iterates (EMA) is a cheap, reliable cure.

You train an imitation policy, watch the behavior-cloning loss slide down and flatten, and conclude that training is healthy. Every signal on your screen agrees. Then you roll the policy out in its environment and measure the thing you actually care about — the long-horizon reward — and it lurches from near-expert to near-failure between one checkpoint and the next. The loss is calm; the reward is chaotic. This paper is about that gap, and about why it is so easy to miss.

The gap matters because modern models live inside feedback loops. A language model's next token depends on the tokens it has already generated; a controller's next observation depends on the action it just took. Yet we almost always train these systems on a surrogate objective — next-token prediction, single-step imitation — that quietly pretends the loop is not there. The surrogate tends to improve smoothly, which lulls us into trusting it. Because rolling out and scoring the true task at every checkpoint is expensive, the swings usually stay invisible, and whichever checkpoint you happen to grab may be a bad one.

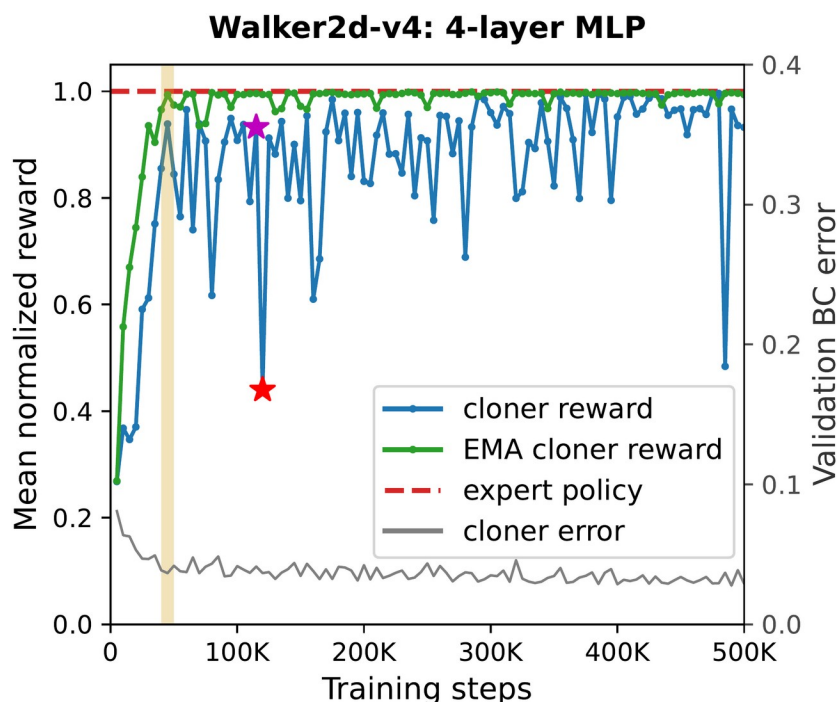


Figure 1. Behavior cloning a Walker2d-v4 expert with a 4-layer MLP over a horizon of $H = 1000$. The rollout reward (blue) swings violently across training iterates, while the single-step BC error (gray) stays flat and low.

An exponential moving average of the weights (green) tracks the expert (red dashed) almost perfectly. The reward you care about is oscillating; the loss you watch never shows it.

The authors argue the culprit is neither the data nor the model, but the noise in the optimizer itself. Minibatch SGD noise, entirely harmless for the one-step training loss, gets amplified through the closed loop into violent swings in long-horizon reward. They name the mechanism gradient variance amplification (GVA), show it is algorithmic rather than statistical, and demonstrate that a nearly free fix — an exponential moving average (EMA) of the weights — tames it across continuous control and autoregressive language models alike.

Paper: <https://arxiv.org/abs/2310.11428> (Block, Foster, Krishnamurthy, Simchowitz, and Zhang; MIT and Microsoft Research NYC).

The loss you watch is not the reward you want

Behavior cloning is the simplest way to imitate an expert: collect demonstrations and minimize a single-step prediction error, the BC loss. In theory, drive that error to zero and you recover the expert policy. In practice, with deep networks and finite compute, the story is stranger. The authors deliberately match the dataset size to the horizon ($N = H = 1000$) so that overfitting cannot be the explanation, and still the rollout reward oscillates hard from iterate to iterate.

Zooming into a handful of consecutive gradient steps makes the point vivid: the reward, viewed as a function of the policy parameters, is a jagged, almost fractal landscape, with good and bad policies sitting right next to each other along stochastic-gradient directions. None of that jaggedness is visible in the one-step BC objective, which is exactly why watching the training loss gives false comfort. The reward and the surrogate have come apart.

Ruling out the usual suspects: GVA is algorithmic, not statistical

There are two candidate explanations for the oscillations. One is statistical — not enough data, or too little model capacity. The other is computational, or algorithmic — something about the optimization process itself. To tell them apart, the authors run an unusually thorough sweep, 278 distinct interventions on the continuous-control task, varying the dataset size (out to effectively infinite data), the architecture (from 2-layer to 6-layer MLPs, plus Transformers), the model scale, and a battery of regularizers such as dropout, weight decay, and augmentation.

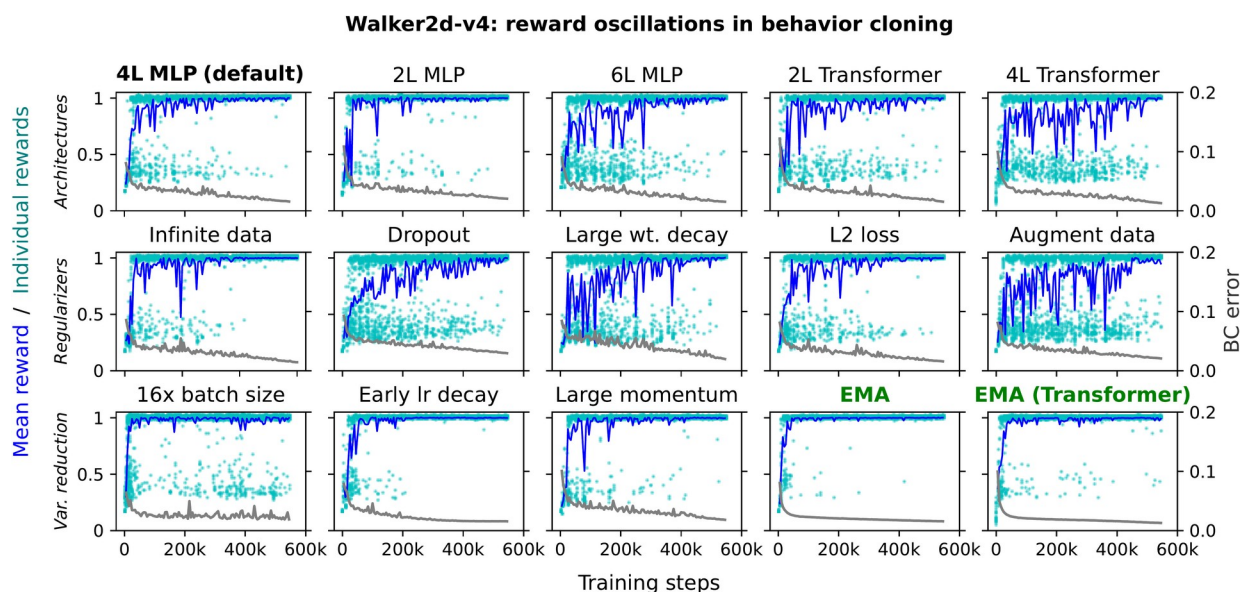


Figure 2. Rollout reward (blue = mean over 20 initial conditions; teal = per-episode) stays jagged as the authors vary architecture, dataset size, model scale, and regularizer, and is calmed only by variance-reduction strategies (bottom row). Standard regularizers do not remove the oscillations and often worsen them — evidence that the instability is algorithmic, driven by SGD noise, rather than statistical.

The pattern in Figure 2 is the whole argument in one image. The oscillations refuse to go away as the data grows, the architecture changes, or regularizers are added — often the regularizers make things worse. They yield only to genuine variance reduction: much larger batches, aggressive learning-rate decay, large momentum, and, most cleanly, iterate averaging. That is precisely the fingerprint you would expect if minibatch gradient noise were the driver, because only reducing that noise should help. The authors call the mechanism gradient variance amplification: SGD noise, negligible for a single step, compounding through the closed loop of the rollout.

The fix: average your iterates

If gradient noise is the disease, averaging is the obvious remedy, and it turns out to be remarkably effective. Keep a running exponential moving average of the weights, updated every step as $\theta \leftarrow (1 - \gamma) \cdot \theta + \gamma \cdot \theta$, and evaluate that averaged model. It costs one extra copy of the parameters, changes nothing about the learning-rate schedule or the batch size, and does not slow down the underlying optimizer.

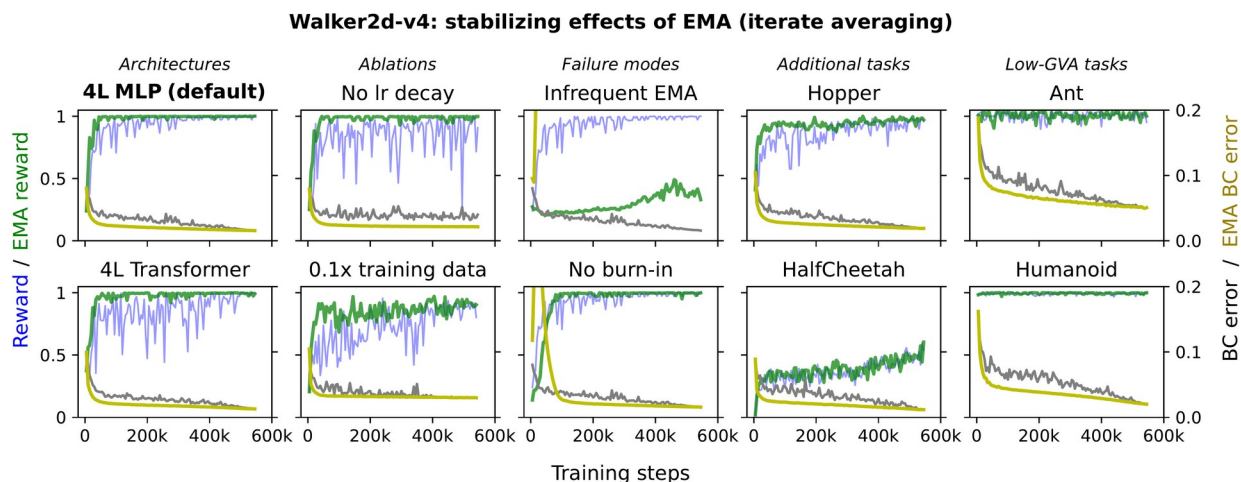


Figure 3. EMA (green) collapses the raw-iterate reward oscillations (faint blue) across architectures, smaller datasets, and additional MuJoCo tasks — Hopper, Ant, HalfCheetah, Humanoid — without touching the learning-rate schedule or the batch size. The recipe matters: update every step with a burn-in and an annealed rate; skipping the burn-in or updating infrequently (center) weakens the effect.

Figure 3 shows the cure holding across architectures, dataset sizes, and other locomotion tasks. The recipe is specific: update the average at every iterate, use an initial burn-in to discard the ragged early weights, and anneal the averaging rate as $\gamma(t) = t^{(-\alpha)}$. Skip the burn-in, or update the average only occasionally, and its stabilizing power fades. There are bonuses too: when data is scarce (down to 0.1x the trajectories), the oscillations are worse and EMA turns that into a real gain in sample efficiency, and EMA can stand in for learning-rate decay altogether.

Table 1. How the tested interventions affect GVA-induced reward oscillations. Only genuine variance reduction helps; iterate averaging delivers the effect at almost no cost.

Strategy	Reduces oscillations?	Cost
Standard regularizers (dropout, weight decay, augmentation)	No — sometimes worse	Low
Aggressive learning-rate decay	Yes	High compute
Large batch (16x via gradient accumulation)	Yes	High compute
Iterate averaging (EMA)	Yes — most robust	Nearly free

The same butterfly effect in language models

GVA is not a quirk of robot control. The authors reproduce the whole story with 270M-parameter (12-layer) Transformers trained on TinyStories, a small synthetic corpus of simple children's tales. Beneath a smooth, monotone validation-loss curve, consecutive checkpoints fluctuate by tiny amounts in next-token loss — and yet their argmax-decoded generations bifurcate into entirely different stories. Adjacent iterates disagree on more than 50% of prompted completions, versus under 5% for EMA iterates.

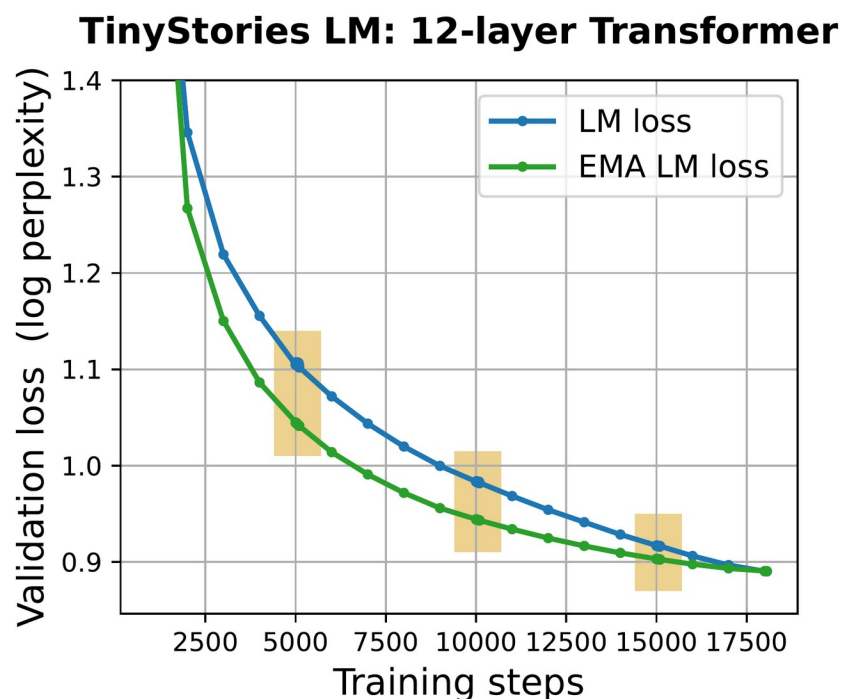


Figure 4. GVA in autoregressive language modeling: a 270M-parameter (12-layer) Transformer on TinyStories. The EMA model's validation loss (green) sits below the raw model's (blue) throughout training and reaches the lowest perplexity. Beneath these smooth curves, nearby training checkpoints generate divergent stories — the same iterate-to-iterate butterfly effect seen in control.

The averaged model does more than stabilize generations. Because EMA decouples optimization from stabilization, it lets you train at a constant learning rate without the usual final decay, which reduces overfitting and yields the lowest-perplexity model in the comparison. The same cheap trick that steadies a walking robot also produces a better language model.

A short theory vignette

To show the amplification is not an artifact of deep networks, the authors add a convex vignette in a simple deterministic control setting with a discontinuous cliff loss. There, a parameter perturbation Δ that changes the BC loss by only order $H \cdot \delta^2$ can change the long-horizon reward gap by a factor that grows like $\exp(H \cdot \delta)$ — exponential in the horizon. An amount of error that is negligible for the one-step objective becomes catastrophic once it compounds over a long rollout. Seen this way, the empirical butterfly effect is not mysterious at all; it is exactly the behavior a closed feedback loop is bound to produce over a long horizon.

What to take away

The headline is simple and actionable. Stochastic gradient noise is invisible to the one-step loss you monitor, but once your model sits inside a feedback loop, that same noise is amplified into chaotic, long-horizon failure — gradient variance amplification. So a flat surrogate loss is not evidence of stable deployed behavior in any closed-loop setting, and iterate averaging

deserves to be a first-class stabilizer alongside your optimizer and your scheduler: cheap, broadly effective, and useful from robot control to language generation.

If you take one habit from this paper, make it this: in any setting where your model acts inside a loop, keep an exponential moving average of the weights, update it every step after a short burn-in, and evaluate that averaged model instead of the latest iterate. It costs almost nothing and rarely hurts, and — as both the walking robots and the TinyStories language models show — it can turn a wildly oscillating training run into a stable, deployable one.

The honest boundary is worth stating. The mechanism is diagnosed empirically, supported by the convex vignette rather than a full theory of deep imitation, and the severity of GVA varies by task — the higher-dimensional Humanoid, for instance, oscillates more benignly. But the practical message survives all of that: when a model acts inside a loop, watch the reward, reduce the variance, and average your weights.